Audit Trail PRD: 21 CFR Part 11-Ready Audit Trail and Audit Log Viewer

Audit Trail PRD

Audience	Management, Product, Engineering, QA, Security, Customer-facing teams
Document owner	@Chris Mills
Status	Draft
Date	4 September 2026
Primary recommendation	Harden the existing PaperTrail versioning into a complete, append-only, retained audit trail (Phase 1), then expose it through an Audit Trail tab in Reports with CSV export and per-record history (Phase 2). Build on what is already in the codebase rather than introducing a new audit store.
Product area	Compliance / Reports / Account Settings

Executive summary

Calira is increasingly sold into regulated laboratories (GLP/GMP, pharma, CRO and contract-manufacturing customers) whose procurement and QA teams ask whether the platform's records are supported by a 21 CFR Part 11-compliant audit trail. Today the answer is "partially": the application already uses the PaperTrail gem to version a small number of models, records login activity via authtrail, and versions custom-form templates. However, coverage is incomplete, background jobs deliberately switch versioning off, the audit data can be modified by the application's own database role, there is no before/after diff, no retention strategy independent of the primary database, and no way for a customer to view or export the trail.

This PRD proposes a two-phase delivery. Phase 1 (MVP) makes the audit trail complete, secure and retained: every create/update/delete on GxP-relevant records is captured with actor, timestamp, before/after values and request context; the audit table is made append-only at the database level; and closed periods are exported to write-once (WORM) storage in S3. A basic filtered CSV export is included so the trail is "available for agency review and copying" from day one. Phase 2 adds the interrogation layer: an Audit Trail tab in Reports with a searchable, filterable viewer, a History tab on individual records, reason-for-change capture on sensitive edits, and a tamper-evidence (hash-chain) verification report. Access is gated to Enterprise accounts and administrator roles; recording itself is enabled for all tiers, because an audit trail cannot be started retrospectively.

1. Problem statement

21 CFR Part 11 §11.10(e) requires "secure, computer-generated, time-stamped audit trails to independently record the date and time of operator entries and actions that create, modify, or delete electronic records", that "record changes shall not obscure previously recorded information", and that the trail be retained for as long as the record and be available for agency review and copying. Customers in regulated environments increasingly make this a hard requirement in RFPs and vendor assessments (see HECVAT, supplier assurance questionnaires and the current RFP set).

The current implementation has the following gaps against that requirement:

- Coverage: only Asset, AssetRequest and ServiceRequest are versioned, and the latter two only on update/destroy and for a handful of attributes. Creates, user/role changes, project changes, custom form responses and account settings are not captured.
- System actions are suppressed: the booking status-transition jobs wrap their updates in PaperTrail.request(enabled: false), so system-generated modifications leave no trace.
- No before/after values: the versions table stores only the prior object snapshot; there is no object_changes diff, no IP address, request id, transaction id or reason for change.
- Not secure: versions is an ordinary table writable (UPDATE/DELETE) by the application's database role. Nothing prevents the application, a bug, or a compromised credential from altering history.
- Not independently retained: audit rows live only in the primary Postgres cluster and its backups; there is no immutable, off-cluster copy with a defined retention period.
- Not interrogable: there is no UI, API or export. Support can only query the trail via a Rails console.
- Legacy login logs (admin_login_logs, department_login_logs) store login_time as a date, not a timestamp; authtrail's login_activities is the reliable source but is not surfaced.

2. Product strategy and phasing

Phase	Release	Scope	Reason
Phase 1	MVP	Complete capture across GxP-relevant models (incl. system actions); before/after diffs and request context; append-only enforcement in Postgres; monthly WORM export to S3 Object Lock; filtered CSV export for admins; validation evidence pack.	Delivers the substantive Part 11 §11.10(e) controls and an honest "yes" in RFPs. Export satisfies "available for review and copying" without waiting for a UI.
Phase 2	Extension	Audit Trail tab in Reports (search, filter, sort, paginate); History tab on bookings, equipment, users and forms; reason-for-change capture on sensitive mutations; hash-chain tamper-evidence with scheduled verification; pgaudit for out-of-application database access; optional read-only Auditor role.	Makes the trail usable day-to-day by customer QA, supports periodic audit-trail review SOPs, and strengthens the "secure" claim with tamper evidence.
Future	Separate PRD	Electronic signatures (§11.50, §11.70, §11.200): signature meaning, manifestation on records, signature/record linking, re-authentication on signing.	Adjacent but distinct scope; depends on Phase 1 being in place.

3. Goals and non-goals

Goals

- Record every create, modify and delete of a GxP-relevant electronic record with: who (user or named system process), when (UTC, millisecond precision), what changed (old and new values), and the request context (IP, user agent, request id, transaction id).
- Guarantee that audit records cannot be updated or deleted through the application or its database credentials, and that changes never obscure previously recorded information.
- Retain audit records for at least as long as the underlying record, in a copy that is independent of the primary database.

- Allow authorised administrators to interrogate the trail by record, actor, event type and date range, and to export the result to CSV.
- Produce validation evidence (requirements, automated tests, tamper tests, test summary) that a customer QA function can reference in their own computerised-system validation.
- Do all of the above by extending the existing PaperTrail, authtrail, GraphQL and Sidekiq foundations rather than adding a new audit subsystem.

Non-goals for Phase 1 MVP

- No interactive audit viewer UI (Phase 2); Phase 1 exposes the trail via CSV export only.
- No electronic signatures, signature meanings or signature manifestations (separate PRD).
- No auditing of read/view events (Part 11 requires create/modify/delete; view auditing is a possible future enhancement for highly sensitive data).
- No changes to customer-side SOPs; Calira provides the capability and evidence, customers own periodic review.
- No blockchain or external ledger service. Amazon QLDB has an announced end of support and is explicitly rejected.

4. Target users and personas

Persona	Need	Phase
Lab Admin / Org Admin	Demonstrate to internal QA and external inspectors that booking, equipment, training and form records have a complete, unaltered history. Export the trail for a given piece of equipment, user or date range.	Phase 1 (export), Phase 2 (viewer)
Customer QA / Compliance reviewer	Perform periodic audit-trail review: who changed what, when and why; spot unusual patterns (e.g., edits to completed bookings, role escalations).	Phase 2
Calira Support / Engineering	Investigate incidents and customer disputes ("who cancelled this booking?") without a Rails console, with an attributable record of system-initiated changes.	Phase 1
Regulatory inspector (indirect)	Receives a human-readable export from the customer during an inspection; needs printed names, UTC timestamps and old/new values.	Phase 1 (via export)
Researcher / Student / Guest	No access to the audit trail. May see a History tab on their own bookings in Phase 2 (product decision, see open questions).	n/a

5. Entitlement and availability

Two distinct decisions apply here and should not be conflated:

- **Recording is universal.** Audit capture runs for every account on every tier from the moment of release. A trail cannot be back-filled, and a customer who upgrades to Enterprise later must not find a gap. Capture cost is a few milliseconds per write and is not user-visible.
- **Access is gated to Enterprise.** The CSV export (Phase 1) and the Audit Trail tab / History tabs (Phase 2) are available only to accounts on the Enterprise tier and only to users with the admin role. Core and Premium accounts see the feature in the locked state per existing gating patterns (add :audit_trail to Subscription::FEATURES[ENTERPRISE_TIER]) to support Enterprise upsell.

Roles: admin only for Phase 1. Phase 2 may introduce an optional read-only auditor role for customer QA staff who should not hold full admin rights (open question).

6. Phase 1 MVP: Complete, secure, retained audit trail

6.1 User value

As a Lab Admin in a regulated lab, I want every change to bookings, equipment, users, roles, projects and custom form responses to be recorded automatically with who, when and what changed, and to be able to export that history for a record or a period, so that I can satisfy my QA function and regulatory inspectors that Calira's records are trustworthy.

6.2 User flow

1. Admin navigates to Reports → Audit Trail (Phase 1 renders a simple export form on this tab; the full viewer arrives in Phase 2).
2. Selects a scope: a record type (Bookings, Equipment, Users & roles, Projects, Custom form responses, Account settings) and optionally a specific record; or an actor; or all.
3. Selects a date range (default: last 90 days).
4. Clicks "Export CSV". The export is generated in the background; the admin receives an email with a time-limited download link when it is ready.
5. The CSV contains one row per audit event: timestamp (UTC), event, record type, record id, record label, actor printed name, actor id, field, old value, new value, reason (if captured), IP address, request id, transaction id.
6. Separately, Calira Engineering can verify integrity (append-only enforcement, WORM copy present) via automated checks; no user action is required.

6.3 Functional requirements

Area	Requirement
Capture — user actions	Every create, update and destroy on the models listed in 6.4, performed through the web UI, the internal GraphQL API, the internal REST API or the external (Doorkeeper) API, produces an audit event with actor, timestamp, old/new values and request context.
Capture — system actions	Changes made by Sidekiq jobs, scheduled tasks, integrations (Benchling, eLabNext, Essentim, Outlook) and data migrations are recorded with a named system actor (e.g., system:AssetRequests::MarkCompletedJob or integration:benchling), never suppressed.
Capture — authentication	Successful and failed logins, lockouts, 2FA enrolment/removal, password changes and SSO logins are recorded (authtrail login_activities plus Devise hooks) and included in exports.
Integrity	Audit rows cannot be updated or deleted by the application database role. Attempts fail at the database and are reported to Sentry. Deleting a record does not delete its audit history.
Timestamping	All audit timestamps are stored as timestamptz in UTC, generated by the database (now()), with cluster nodes NTP-synchronised. Timestamps are never supplied by the client.
Retention	Audit rows for a closed calendar month are exported to S3 with Object Lock (compliance mode) and a retention period of at least 10 years (configurable). Postgres retains all rows online for a minimum of 7 years before any archival policy applies.
Export	Admins on Enterprise accounts can export a filtered CSV. Exports are generated asynchronously, stored in S3, delivered by a signed URL valid for 24 hours, and the export itself is audited.

Area	Requirement
Access control	Export is limited to admin role on Enterprise accounts via Pundit and Subscription::FEATURES. Actor names resolve to printed names (§11.50-style), not ids.
Validation	A validation pack is produced: requirements trace matrix, automated test evidence (RSpec), tamper-attempt test evidence, and a signed test summary, stored with SDLC evidence in GitLab/TestRail.

6.4 Records in scope (MVP)

Model (app-clustermarket)	Why it is a GxP-relevant record	Current state
AssetRequest, ServiceRequest, BookingSet, BookingInvitation, RequestProject, AssetRequestTransaction	Usage and maintenance bookings are the primary evidence of equipment use, calibration and maintenance.	Partial (update/destroy, few attributes)
Asset, AssetMaintenanceContract, AssetTraining, AssetsCustomForm	Equipment master data, maintenance contracts and training records.	Asset only
CustomFormInput (form responses), CustomForm, CustomFormHistory	Captured supporting data (calibration results, incident reports, hazard declarations).	Templates versioned via CustomFormHistory; responses not audited
User, AccessRole, TwoFactorAuth, WebauthnCredential, Identity	Who can do what; role escalation and credential changes.	Not audited
Project, Fund, FundingCode, Group	Attribution of usage to projects/funding.	Not audited
Account, Department, Subscription, Feature, Integration, IntegrationOauth	Account-level configuration that changes system behaviour (feature toggles, integration connections).	Not audited
Workflow and workflow steps	Configured approval logic.	Not audited

6.5 UI changes for MVP

- Add an "Audit Trail" tab to the existing Reports page (AdvancedReportsIndex.svelte), visible to admins; locked state for non-Enterprise accounts.
- Tab content for Phase 1: scope selector (record type / record / actor), date range, "Export CSV" button, and a list of previous exports with status and download link.
- Toast/email on export completion.

6.6 MVP high level acceptance criteria

- For every model in 6.4, creating, updating and deleting a record through the UI, GraphQL, internal REST and external API each produce exactly one audit event per transaction with correct actor, UTC timestamp, and old/new values for changed attributes only.
- Booking status transitions performed by Sidekiq jobs produce audit events attributed to the named job.
- An UPDATE or DELETE statement against versions executed with the application's database credentials fails; the failure is visible in Sentry.
- Destroying an audited record leaves its audit history intact and queryable by item type/id.
- Closed-month partitions are present in the S3 Object Lock bucket, with a retention date set, and cannot be deleted by the application's IAM role.

- An Enterprise admin can export a CSV for a date range and record type; a Premium admin sees the locked state; a researcher cannot reach the tab or the mutation.
- The export is itself recorded as an audit event (who exported what, when).
- p95 latency of audited write requests increases by no more than 10 ms compared with the pre-release baseline.
- Validation pack reviewed and signed off by Engineering, QA and Security prior to release, per the SDLC approval matrix (high-risk change).

7. Phase 2 extension: Interrogation, tamper evidence and reason for change

7.1 User value

As a Lab Admin or QA reviewer, I want to browse and search the audit trail in the product, open the history of any booking, piece of equipment or user directly from its page, require a reason when sensitive records are edited, and prove that the trail has not been tampered with, so that periodic audit-trail review is practical and inspections are straightforward.

7.2 Extension functional requirements (out of scope for MVP)

Area	Requirement
Audit Trail viewer	Reports → Audit Trail shows a paginated, sortable table of events filterable by record type, record, actor, event, field and date range; row expands to show old/new values and context. Uses the same table component as the equipment overview.
Per-record History	Booking, Equipment, User and Custom form pages gain a History tab listing that record's events, with a link to the full viewer.
Reason for change	Configurable per account: editing or cancelling a completed/past booking, changing a user's role, and editing a submitted form response prompts for a reason, stored on the audit event.
Tamper evidence	Each audit row carries a SHA-256 hash chained to the previous row; a nightly job verifies the chain and the S3 copy and records the result. Admins can download a verification report.
Out-of-application access	pgaudit is enabled on the CloudNativePG cluster for DDL and writes to audit tables; logs ship to Coralogix and S3 so direct database access by operators is itself audited.
Auditor role (optional)	A read-only auditor role can access the Audit Trail tab and exports without other admin powers.
Export formats	PDF export of a filtered view with page headers, filter criteria and generation timestamp, for inclusion in inspection binders.

8. Technical design and implementation

This section is engineering-level and is intended to seed refinement. All paths are relative to the app-clustermarket repository. The guiding principle is to extend what exists: PaperTrail 15 with paper_trail-globalid, authtrail, Sidekiq, the graphql-ruby schema (Relay connections, Pundit integration, Dataloader), CloudNativePG on EKS, and S3.

8.1 Current state (as-is)

- `Gemfile:83-84` paper_trail ~> 15 and paper_trail-globalid; whodunnit stored as a GlobalID string, resolved via version.actor.

- `app/controllers/application_controller.rb:15` before_action :set_paper_trail_whodunnit with user_for_paper_trail → current_user; inherited by GraphQLController and internal API controllers.
- `db/structure.sql:4384` versions(id, item_type, item_id, event, whodunnit, created_at timestamp without time zone, object jsonb). No object_changes, no metadata columns, single index on (item_type, item_id).
- `app/models/asset.rb:50, asset_request.rb:58, service_request.rb:29` the only has_paper_trail declarations; the latter two restricted with on:/only:.
- `app/jobs/asset_requests/mark_completed_job.rb:10 (and mark_inprogress_job, service_requests/mark_completed_job)` PaperTrail.request(enabled: false) around status updates.
- `app/models/asset_request/bulk_insert.rb:131–147` manual PaperTrail::Version.insert_all for bulk-created bookings — must be kept in step with any new columns.
- `authtrail (Gemfile:141), login_activities table` already records scope, strategy, success, failure_reason, ip, user_agent, created_at. Reuse as-is; deprecate admin_login_logs / department_login_logs.
- `app/models/custom_form_history.rb` snapshot versioning of form templates with user and version_number. Keep; surface in the viewer alongside versions.
- `Database` PostgreSQL via CloudNativePG in-cluster (per the production architecture overview), Barman backups to S3, daily backups + continuous PITR. Not RDS: RDS-only features do not apply, but the CNPG images ship pgaudit.

8.2 Architecture overview

Capture stays in-process (ActiveRecord callbacks via PaperTrail) so that the audit write is in the same database transaction as the change it records — an audit row can never exist without its change or vice versa. Integrity is enforced in Postgres, independent of Ruby. Durability is achieved by exporting closed partitions to S3 Object Lock. Interrogation is served by the existing GraphQL schema and a Sidekiq-backed export job.

```
Web / GraphQL / REST / External API       Sidekiq jobs & integrations
        |  (whodunnit = user GID,                 |  (whodunnit = 'system:JobName' /
        |   ip, user_agent, request_id)           |   'integration:benchling')
        v                                         v
  ActiveRecord model  --has_paper_trail-->  versions (partitioned, append-only)
                                                  |   ^ REVOKE UPDATE/DELETE
                                                  |   ^ BEFORE UPDATE/DELETE trigger RAISE
                                                  |   ^ (Phase 2) BEFORE INSERT hash-chain
trigger
                                                  v
   GraphQL auditTrail connection  <-- Pundit (admin + Enterprise) --> Reports > Audit Trail
   AuditTrail::ExportJob  --> CSV in S3 (signed URL, 24h)
   AuditTrail::ArchiveJob (monthly) --> S3 Object Lock bucket (compliance mode, 10y)
   (Phase 2) AuditTrail::VerifyChainJob (nightly) --> Sentry alert / verification report
   pgaudit (CNPG) --> fluentd --> Coralogix + S3 (out-of-application DB access)
```

8.3 Data model changes

Extend the existing versions table rather than introduce a new one, so existing rows remain valid and PaperTrail's own APIs (reify, version_at, changeset) keep working.

```
# db/migrate/xxxx_harden_versions_for_audit_trail.rb
add_column :versions, :object_changes, :jsonb          # {attr => [old, new]}; the Part 11
payload
add_column :versions, :transaction_id, :bigint         # PaperTrail groups multi-row changes
add_column :versions, :ip, :inet
add_column :versions, :user_agent, :string
add_column :versions, :request_id, :uuid               # ties to lograge / Sentry
add_column :versions, :actor_label, :string            # printed name at time of event (Part
11.50 style)
add_column :versions, :account_id, :bigint             # tenancy scope for querying / export
```

```
add_column :versions, :reason, :text                    # Phase 2: reason for change
add_column :versions, :row_hash, :bytea                 # Phase 2: hash chain
change_column_default :versions, :created_at, -> { 'now()' }
change_column :versions, :created_at, :timestamptz, null: false
add_index :versions, :transaction_id
add_index :versions, [:account_id, :created_at]
add_index :versions, :whodunnit
add_index :versions, :created_at
add_index :versions, :object_changes, using: :gin, opclass: :jsonb_path_ops  # optional,
field-level search
```

Partitioning: convert versions to a range-partitioned table by created_at (monthly) using the standard create-new-table / copy / rename approach in a maintenance window, or pg_partman if adopted for other tables. Partitioning is what makes monthly archival, retention and index bloat manageable; it is much cheaper to do now than after several years of growth. account_id is denormalised onto the row (from the item's account, resolved in a PaperTrail meta callback) so tenant-scoped queries and exports do not need joins.

PaperTrail serialisation: keep jsonb for object and object_changes. Sensitive attributes (encrypted_password, otp_secret, tokens, API keys) are excluded via the model's paper_trail skip: list; Rails' filter_parameters list is applied to object_changes in a PaperTrail serializer wrapper as a second line of defence.

8.4 Capture

Models

Introduce app/models/concerns/auditable.rb so coverage is declarative and greppable:

```
module Auditable
  extend ActiveSupport::Concern
  included do
    has_paper_trail versions: { class_name: 'AuditTrail::Version' },
                    meta: { account_id: :audit_account_id, actor_label: ->
{ PaperTrail.request.controller_info&.dig(:actor_label) } },
                    skip: self::AUDIT_SKIP_ATTRIBUTES
  end
end
```

Apply Auditable to every model in 6.4, removing the existing on:/only: restrictions on AssetRequest and ServiceRequest. AuditTrail::Version < PaperTrail::Version adds scopes (for_account, for_item, by_actor, between), the actor printed-name resolver, and the CSV row presenter. The bulk_insert.rb path is updated to populate the new columns.

Request context

```
# ApplicationController (also inherited by GraphQLController, Api::*,
ExternalApi::BaseController)
def info_for_paper_trail
  { ip: request.remote_ip, user_agent: request.user_agent, request_id: request.request_id,
    actor_label: current_user&.name }
end
```

ExternalApi::BaseController authenticates via Doorkeeper rather than Devise; set PaperTrail.request.whodunnit from doorkeeper_token.resource_owner and record the OAuth application name in actor_label ("Jane Doe via Benchling App").

System actions

Replace every PaperTrail.request(enabled: false) with attribution, and add a Sidekiq server middleware so all jobs are attributed by default:

```
# app/lib/sidekiq/audit_context_middleware.rb
```

```
class Sidekiq::AuditContextMiddleware
  def call(worker, job, _queue)
    PaperTrail.request(whodunnit: job['audit_whodunnit'] || "system:#{worker.class.name}",
                       controller_info: { actor_label: job['audit_actor_label'] ||
worker.class.name }) { yield }
  end
end
# Client middleware copies the current whodunnit into job['audit_whodunnit'] when a job is
enqueued
# from a request, so user-initiated async work stays attributed to the user.
```

Integrations use the same mechanism with whodunnit: "integration:#{name}". Data migrations and rake tasks run under "system:migration:<name>". Enabling PaperTrail is the default; disabling it requires a code-reviewed justification (Rubocop custom cop flags PaperTrail.request(enabled: false)).

Authentication events

authtrail already records login attempts. Add Devise/Warden hooks (after_set_user, before_logout) and model callbacks on TwoFactorAuth, WebauthnCredential and password changes to write AuditTrail::Version rows with events login, logout, password_changed, two_factor_enabled/disabled, locked, so that a single table answers all questions in Phase 2. login_activities remains the source for failed-attempt geodata.

8.5 Integrity (append-only at the database)

```
-- Executed as the CNPG superuser in a migration using execute; app role name per CNPG
cluster config
REVOKE UPDATE, DELETE, TRUNCATE ON versions FROM app_role;   -- and on each partition; set
default privileges
CREATE OR REPLACE FUNCTION versions_immutable() RETURNS trigger AS $$
BEGIN RAISE EXCEPTION 'versions is append-only (audit trail)'; END $$ LANGUAGE plpgsql;
CREATE TRIGGER versions_no_update BEFORE UPDATE OR DELETE ON versions
  FOR EACH ROW EXECUTE FUNCTION versions_immutable();
```

Consequences to design around: PaperTrail's item association uses dependent: :destroy by default — set has_paper_trail ... versions: { dependent: nil } (or rely on the trigger, but do not generate the DELETE). Retention deletions are performed only by detaching whole partitions under a documented, dual-approved procedure after the WORM copy is verified — never by row deletes. Schema migrations that need to touch versions run as the migration role, which is separate from the application role in CNPG.

Phase 2 hash chain: a BEFORE INSERT trigger sets row_hash = sha256(prev.row_hash || canonical_json(NEW)) per account (chain per account_id keeps verification parallelisable and tenant-scoped). AuditTrail::VerifyChainJob walks yesterday's rows per account, compares against the S3 copy's manifest, and writes a verification event; a break pages on-call via Sentry/Grafana OnCall.

8.6 Retention and independent copy

- New S3 bucket (e.g. clustermarket-audit-archive-<env>) with Object Lock enabled in compliance mode, default retention 10 years, versioning on, SSE-KMS with a dedicated key, bucket policy denying s3:DeleteObject to all principals; the application IAM role has PutObject/GetObject only.
- AuditTrail::ArchiveJob (sidekiq-scheduler, runs on the 2nd of each month): streams the previous month's partition to gzipped NDJSON in S3, writes a manifest (row count, min/max id, SHA-256 of the file) and records an archived event in versions. Idempotent; verifies by re-reading the object and comparing the hash.
- Turn on Object Lock (governance mode, 35 days) for the Barman backup bucket so PITR chains cannot be deleted during the retention window; this is an infra change independent of the application.

- Retention policy: online in Postgres ≥ 7 years; S3 ≥ 10 years or per contract. Documented in the Information Asset Policy; detaching a partition requires the S3 manifest to be verified first.

8.7 Query API and export

GraphQL additions follow the existing base-class conventions (Types::BaseObject, BaseConnection, Pundit roles, Dataloader Sources):

```
# app/graphql/types/audit_trail/entry_type.rb
class Types::AuditTrail::EntryType < Types::BaseObject
  pundit_role :audit_trail_reader
  field :id, ID, null: false
  field :occurred_at, GraphQL::Types::ISO8601DateTime, null: false
  field :event, Types::Enums::AuditTrail::Event, null: false   #
create/update/destroy/login/...
  field :item, Types::AuditTrail::ItemUnion, null: true         # resolves via GlobalID; null
if destroyed
  field :item_type, String, null: false
  field :item_label, String, null: true                          # e.g. 'Booking #1234 –
Confocal 2'
  field :actor, Types::UserType, null: true                      # Dataloader Source batches
GID lookups
  field :actor_label, String, null: false                        # printed name / system:Job
  field :changes, [Types::AuditTrail::FieldChangeType], null: false  # {field, old, new}
  field :reason, String, null: true
  field :ip, String, null: true
  field :request_id, String, null: true
  field :transaction_id, ID, null: true
end

# Types::QueryType
field :audit_trail, Types::AuditTrail::EntryType.connection_type, null: false do
  argument :filter, Types::Inputs::AuditTrail::Filter, required: false
  # filter: item_type, item_id, actor_id, events, field, from, to, search
end
# Types::AssetType / AssetRequestType / UserType / CustomFormType
field :history, Types::AuditTrail::EntryType.connection_type, null: false  # Phase 2

# Mutations::AuditTrail::RequestExport  -> enqueues AuditTrail::ExportJob, returns export id
# Mutations::AuditTrail::* mutations that edit sensitive records gain `reason: String` (Phase
2)
```

Authorization: AuditTrailPolicy#index? requires admin role on current_account and account.subscription.feature_enabled?(:audit_trail). Resolvers always scope by account_id (denormalised column) and never trust item ids from the client without the account scope. Cursor pagination via BaseConnection with default_max_page_size 100 already set on the schema.

Export: AuditTrail::ExportJob builds the CSV with the same filter input, streaming in batches of 5,000 rows, writes to the exports S3 bucket (30-day lifecycle), emails a signed URL (24 h), and records an export event including the filter used. Exports over 1M rows are chunked into multiple files in a zip.

8.8 Frontend

- Reports → new AuditTrail tab within app/javascript/components/Layout/AdvancedReports/, gated by featureFlags/subscription tier with the existing locked-state pattern.
- Phase 1: scope + date range form, Export button, exports list (polls the RequestExport status field).
- Phase 2: results table reusing the equipment overview table component (sort, filter, column chooser), expandable rows for changes, actor and record links; History tab component mounted on booking, equipment, user and custom form pages; reason-for-change modal reused across the affected mutations.
- Apollo codegen picks up the new types from schema.graphql; no new client dependencies.

8.9 Validation, testing and evidence

- Requirements trace: each functional requirement in 6.3/7.2 mapped to RSpec examples (spec/models/concerns/auditable_spec.rb, spec/graphql/queries/audit_trail_spec.rb, spec/jobs/audit_trail/*_spec.rb, spec/requests/external_api/v1/audit_spec.rb) and to TestRail cases for manual/UI checks.
- Coverage guard: a spec that iterates every ApplicationRecord descendant and asserts that models on the 6.4 list include Auditable, and that no model calls has_paper_trail with on:/only: restrictions.
- Tamper tests: specs that attempt UPDATE/DELETE on versions via the app connection and assert PG::InsufficientPrivilege / trigger exception; a spec that destroys an audited record and asserts its versions survive; (Phase 2) a spec that mutates a row as superuser in a transaction and asserts VerifyChainJob detects the break.
- Attribution tests for each Sidekiq job that writes to audited models, and for each integration callback.
- Performance: benchmark spec on the booking write path; CI fails if median added latency > 10 ms. Load test the export job against a 5M-row versions table in staging.
- Time source: cluster nodes on NTP (EKS default via chrony); documented in the validation pack. All timestamps db-generated.
- Evidence pack: URS/FS (this PRD's sections 6–8), trace matrix, CI test report, tamper test output, load test report, security review record, and release approval per the SDLC high-risk approval matrix (QA + Engineering + Security + Leadership).
- Ongoing: the audit trail's own schema and code changes are covered by the SDLC §11.10(k) requirement (change control with traceable history) via GitLab.

8.10 Key decisions and alternatives considered

Decision	Chosen	Alternatives and why not
Audit library	Keep PaperTrail 15 and extend (object_changes, meta, transaction_id are native features).	audited gem: closer fit out of the box (remote_address, request_uuid, comment) but requires migrating existing versions rows and re-testing bulk_insert; no functional advantage. logidze: fast trigger-based JSONB history stored on the record row — wrong shape for an independent trail and loses history on delete without extra work.
Where integrity is enforced	Postgres: REVOKE + trigger; hash chain in Phase 2.	Application-only guards are bypassable by any code path or bug. External append-only ledger (QLDB) is end-of-life; a separate audit database doubles operational burden without removing the need for DB-level controls.
Independent copy	S3 Object Lock, compliance mode, monthly partition export with manifests.	Rely on Barman backups only: backups are deletable by operators and are not a queryable archive. Streaming CDC (Debezium) to S3: more moving parts than a monthly batch for no compliance benefit.
Capture mechanism	In-transaction ActiveRecord callbacks (PaperTrail).	Database triggers writing audit rows: language-agnostic and catches raw SQL, but loses actor/request context unless it is injected via SET LOCAL; adopt only if a significant non-

Decision	Chosen	Alternatives and why not
		ActiveRecord write path emerges. pgaudit is used for operator access, not application events.
Table layout	Single partitioned versions table with denormalised account_id.	Per-model audit tables: simpler queries per model but N tables to protect, archive and verify. Per-account tables: unnecessary at current tenant counts.
Recording scope by tier	Record for all tiers; gate access to Enterprise.	Recording only for Enterprise creates unrecoverable gaps on upgrade and complicates support investigations.

9. Technical considerations translated for product/management

Topic	Product meaning
Append-only database table	Once written, an audit entry cannot be changed or removed even by our own application. This is the single most important control for Part 11 and the one inspectors probe first.
System actions are attributed, not hidden	When Calira itself changes a record (e.g., auto-completing a booking), the trail shows that a named system process did it. Today those changes are invisible.
Before/after values	The trail will show what a field was and what it became, not just that "something changed". This is what makes the export useful in an inspection.
Write-once copy in S3	Every month, the audit data is copied to storage that cannot be deleted for ten years, even by an administrator. This protects the trail if the database is lost or compromised.
Partitioning	An internal database layout change that keeps the audit table fast and makes archiving cheap. Requires a short maintenance window; no user-visible change.
Validation pack	A bundle of evidence (requirements, tests, sign-offs) customers can cite in their own system validation. It is a deliverable, not just an internal QA activity.
Recording for all tiers, viewing for Enterprise	Everyone's history is kept from day one so upgrading customers have a complete trail; only Enterprise admins can see or export it.

10. Risks and mitigations

Risk	Impact	Mitigation
Write-path latency and table growth	Auditing every model adds a row (or several) per write; versions becomes the largest table.	Batch-friendly jsonb columns, monthly partitions, targeted indexes only, GIN index optional; benchmark gate in CI; archive job keeps online retention bounded.
Breaking existing code that expects to delete versions	dependent: :destroy on versions, admin tooling or specs that clean up versions would now fail.	Set dependent: nil; grep and update specs/factories; run the full suite against a database with the REVOKE applied in CI.
Sensitive data captured in the trail	Passwords, OTP secrets, tokens or personal data end up in object_changes.	Explicit skip: lists per model; Rails filter_parameters applied in the serializer; spec

Risk	Impact	Mitigation
		asserting filtered keys never appear; encryption at rest already in place.
Partition migration on a live table	Converting versions to a partitioned table locks or takes time on a large table.	Create the partitioned table alongside, backfill in batches during off-peak, swap names in a short maintenance window; rehearse in staging with a production-size copy.
Operator bypass	A DBA with superuser access could alter history outside the application.	pgaudit on the cluster shipped to Coralogix/S3 (Phase 2); hash chain verification; S3 compliance-mode copy; RBAC on cluster access per SDLC deployment controls.
Scope creep into e-signatures	Customers conflate audit trail with electronic signatures; delivery stalls.	Explicit non-goal; separate PRD; sales enablement on the distinction.
Exports exposing data to wrong roles	Incident reports and personal data leak via export links.	Pundit + tier gating; signed URLs 24 h; export events audited; exports bucket lifecycle 30 days.

11. Success metrics

- Compliance: 100% of models in 6.4 audited with zero suppressed system paths (measured by the coverage guard spec) at release; zero integrity-check failures in the first 6 months.
- Commercial: audit trail cited as a requirement in RFPs is answered "Yes, fully"; at least 2 Enterprise deals in the first 12 months where Part 11 support was a stated decision factor.
- Adoption: 50% of Enterprise accounts run at least one export or viewer session within 90 days of Phase 2 release.
- Support: 75% reduction in Rails-console investigations of "who changed X" tickets.
- Performance: write-path p95 regression ≤ 10 ms; audit export for 90 days on the largest account completes in under 5 minutes.

12. Rollout plan

Step	Description
Spike & design review	Confirm CNPG role separation and superuser migration path; rehearse partition conversion on a staging copy; agree retention numbers with Security and Legal.
Internal QA	Full regression in staging with REVOKE and trigger active; tamper tests; performance benchmarks; export validation on production-size data.
Security review & validation sign-off	Review of skip lists and export access; validation pack assembled and signed (high-risk change per SDLC approval matrix).
Release Phase 1 (dark)	Capture, integrity and archive enabled for all accounts behind a feature flag for the export UI; monitor write latency and table growth for two weeks.
Release Phase 1 (GA)	Enable the Audit Trail tab export for Enterprise admins; customer-facing documentation and a Part 11 statement published; sales enablement.
Phase 2	Viewer, History tabs, reason for change, hash chain, pgaudit, optional auditor role, PDF export, sequenced by customer feedback.

13. Indicative effort from spikes

To be completed after spikes and story refinement. Estimates are development-only unless stated.

Area	Development estimate	QA estimate
Backend: data model, partitioning, integrity controls		
Backend: capture coverage (Auditable, jobs, integrations, auth events)		
Backend: archive job, S3 Object Lock, IAM/infra		
Backend: GraphQL query, export job		
Frontend: Reports tab (Phase 1 export)		
Validation pack and tamper/performance evidence		
Total Phase 1 MVP		
Phase 2: viewer, History tabs, reason for change		
Phase 2: hash chain, verification, pgaudit		
Total Phase 2		

14. Recommended story split

Phase 1 MVP stories

- BE: Migrate versions — new columns, timestamptz, indexes, partitioning; update bulk_insert path.
- BE: Auditable concern and AuditTrail::Version; apply to all in-scope models; remove on:/only: restrictions; skip lists and filter_parameters serializer.
- BE: Request context (info_for_paper_trail) across ApplicationController, GraphQL, internal API and External API (Doorkeeper actor).
- BE: Sidekiq audit-context middleware; remove PaperTrail.request(enabled: false); integration and migration attribution; Rubocop cop.
- BE: Authentication events (Devise/Warden hooks, 2FA/WebAuthn/password callbacks).
- BE/Infra: REVOKE + immutability trigger via superuser migration; dependent: nil; CI runs suite with REVOKE applied.
- Infra: S3 audit archive bucket with Object Lock, KMS, IAM; Barman bucket Object Lock.
- BE: AuditTrail::ArchiveJob with manifests and self-verification; scheduler entry.
- BE: GraphQL auditTrail connection (filter input, policy, Dataloader actor source) and RequestExport mutation; AuditTrail::ExportJob with signed URL email.
- FE: Reports → Audit Trail tab (Phase 1 export form, exports list, locked state).
- QA/Eng: coverage guard spec, tamper tests, attribution tests, performance benchmark, validation pack.
- Product/Docs: Part 11 capability statement, help-centre article, sales enablement.

Phase 2 stories

- FE: Audit Trail viewer table with filters, expandable changes, links.
- FE/BE: History tab on booking, equipment, user and custom form pages (history field on types).
- BE/FE: Reason-for-change argument on sensitive mutations and modal; account-level setting.

- BE: Hash-chain trigger, AuditTrail::VerifyChainJob, verification report download.
- Infra: pgaudit on CNPG, log routing to Coralogix and S3.
- BE/FE: Optional auditor role in AccessRole and policies; PDF export.

15. Open questions

- Retention numbers: are 7 years online / 10 years WORM acceptable to Legal and to the largest regulated customers, or should retention be configurable per account?
- Should researchers see a History tab on their own bookings (transparency) or is all audit access admin-only?
- Is a dedicated read-only auditor role required for Phase 2, or is admin sufficient for target customers?
- Which mutations require a reason for change by default (proposal: edits/cancellations of past bookings, role changes, edits to submitted form responses)?
- Does any customer contract require audit data residency in a specific region (affects the S3 archive bucket location)?
- Should the external API expose the audit trail (read-only) for customers who integrate with their own QMS?